

Arc Pump Agent

An autonomous AI agent that operates a USDC-native token launchpad on Arc. Every few hours it reasons with Claude about its market, decides to launch a token, buy a bonding curve, or claim creator fees, signs the transaction through a Circle Programmable Wallet, and settles it in USDC on Arc — with no human in the loop. A live dashboard streams every decision and its reasoning.

Track Track 4 — Best Agentic Economy Experience on Arc

Circle Developer account sieusayza@gmail.com

Live demo <https://agent.arcpump.com>

Demo video <https://youtu.be/mK7tNaC4z14>

Code repository github.com/PHUOCHAU2403/arc-pump

Products used on Arc

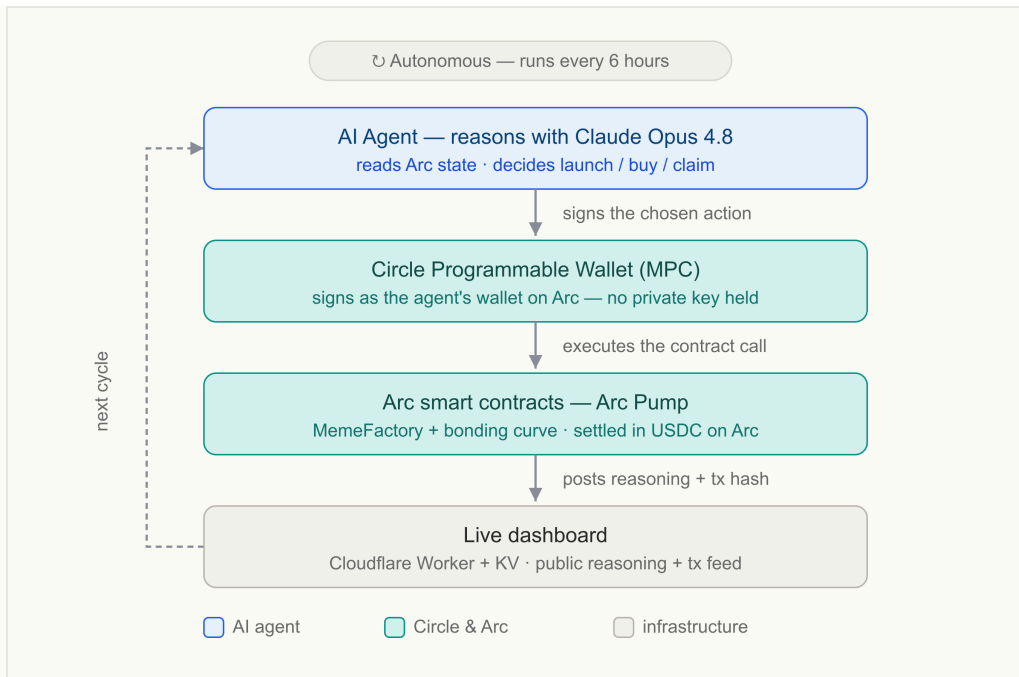
USDC	Programmable Wallets	Nanopayments
Native settlement — launch fees, curve buys, creator-fee payouts	Developer-Controlled (MPC) — the agent's autonomous signer	Next step for high-frequency, sub-cent agentic settlement (conceptual)

Working MVP

The agent is live and autonomous on Arc testnet. It runs unattended on a 6-hour schedule: read Arc state → reason with Claude (Opus 4.8) → decide an action → sign via Circle Programmable Wallet → execute the contract call → post the reasoning + tx hash to a public dashboard. Real launches and buys are confirmed on-chain.

- **Live dashboard:** agent.arcpump.com — streams each decision, its reasoning, and the Arcscan tx link
- **Agent wallet (Circle MPC):** `0x9f26dfba277afdd6e5df307f7d9363abe2f72b6a` — [on Arcscan](#)
- **MemeFactory contract:** `0x4dCf3238dd90E571e82bC07fD876B384f170546c` — [on Arcscan](#) (Arc testnet, chain 5042002)
- **Stack:** Claude API (reasoning) · Circle `@circle-fin/developer-controlled-wallets` (signing) · `viem` (Arc reads) · Cloudflare Worker + KV (dashboard API) · Next.js on Vercel (dashboard UI)

Architecture



Autonomous loop — every 6 hours. AI agent reasons → Circle MPC wallet signs → Arc contracts execute in USDC → dashboard streams reasoning + tx. Then repeats.

How Circle tools are integrated

The agent needs to sign transactions autonomously, on a schedule, with no human approving each one — and without holding a raw private key on a server. **Circle Programmable Wallets (Developer-Controlled, MPC) on ARC-TESTNET** were the natural fit: Circle's MPC holds the key material, and the agent calls the API to execute contract calls. Setup is a handful of SDK calls — register the entity secret, create a wallet set, create an EOA wallet on ARC-TESTNET. Each action runs through `createContractExecutionTransaction` with `abiFunctionSignature` + `abiParameters` + a native amount, calling the payable `MemeFactory.createToken(...)` and `BondingCurve.buy(...)` directly. On Arc, USDC is the native gas/value token, so every action settles in USDC end-to-end — no wrapper, no separate quote token.

Circle Product Feedback

What worked well

- **Dev-controlled wallet setup is genuinely fast** — a few `@circle-fin/developer-controlled-wallets` SDK calls and we had a working, MPC-secured agent wallet on Arc.
- **Contract execution with native value is clean** — `createContractExecutionTransaction` with a native amount let the agent call payable factory/curve methods directly; polling `getTransaction` to `COMPLETE` and reading `txHash` was simple and reliable. Real launches and buys confirmed on the first working attempt.
- **MPC signing "just works" for an autonomous loop** — once set up, the agent signs every 6 hours with no interactive step. Exactly the point of an agentic

wallet.

What could be improved

1. **Entity-secret registration:** `recoveryFileDownloadPath` **must be a directory, not a file path.** Passing `./recovery.dat` throws `Invalid Directory` — it has to be `./`. The error fires after the ciphertext is computed and doesn't make the directory-vs-file requirement obvious.
2. **No working testnet faucet through the Wallets SDK on Arc.**
`requestTestnetTokens({ blockchain: "ARC-TESTNET" })` returned `403 Forbidden` with a Standard API key, so we funded the agent manually. A programmatic fund path would remove a manual onboarding step for a self-sufficient agent.
3. **USDC on Arc is the native token at 18 decimals — and that surprises you.** Most developers' mental model is the 6-decimal ERC-20. On Arc it's the native gas/value token at 18 decimals, and the execution `amount` maps to `msg.value`. Our first payable call under-sent value and reverted before we traced it to the decimals/native-value model.
4. `accountType` **guidance for agents.** EOA worked perfectly; SCA is also offered. For an autonomous agent on Arc it wasn't clear which to prefer or the gas/UX trade-offs — a one-liner in the docs would help.

Recommendations

- Document the entity-secret **directory** requirement in the quickstart, and make the SDK error say "expected a directory path" explicitly.
- Ship a **programmatic Arc testnet faucet** (via the SDK or a documented endpoint) so agents can self-fund in dev — or at minimum return a clearer error pointing to the manual faucet.
- Add an **"agent wallet on Arc" quickstart**: create → fund → execute a payable contract call settling in native USDC. That exact path is the canonical agentic-economy flow and deserves a first-class guide.
- Call out **native-USDC (18-dec) vs standard/bridged USDC (6-dec)** prominently across Arc docs and SDK examples, with a worked payable-call example showing how `amount` becomes `msg.value`.
- **Nanopayments for agents** — a documented, generally-available sub-cent / streaming primitive on Arc would let agents settle per-inference and per-action flows natively.

Overall: Developer-Controlled Wallets + USDC on Arc made a genuinely autonomous, key-safe on-chain agent straightforward to build. The rough edges were all onboarding/docs, not the core product — once past them, the agent has been signing and settling on Arc on its own every 6 hours.